

Python 基础与 Skills 开发实战

第2周 · 第1课 · 唯一一堂语法课

120min

课时长度

8

知识块

30min

Skills 实战

从「读懂 Python 代码」到「亲手开发一个 Skills 技能包」

读懂 · 会问 · 能开发

变量 · 类型 · 条件 · 循环 · 列表 · 字典 · 函数 · 类 · Skills

一、导入 · 这节课学什么

为什么 Python 代码到处都是？

- 后端开发：FastAPI / Django / Flask
- AI Agent 框架：LangChain / CrewAI
- 数据分析 / 机器学习（第3周）
- 爬虫、自动化脚本

**本课目标 = 看到一段 Python 代码，
能大致说出它「在干什么」。**

四个「能回答」的问题

- ① 这段代码在干什么？
- ② 数据从哪来、到哪去？
- ③ 哪里是核心逻辑？
- ④ 想改一个功能，该改哪里？

读代码 ≠ 写代码

读代码只需写代码 30% 的知识量

时间分配总览 · 8 个知识块

段	内容	分钟
一	导入与本课目标	8
二	环境准备 + Hello World	10
三	变量 / 类型 / 条件	20
四	循环 / 列表 / 字典	20
五	函数与模块	15
六	面向对象：看懂「类」	15
七	Skills 开发实战（重点）	30
八	总结 / 作业 / FAQ	2

60min

语法快览（三+四+五）

30min

Skills 实战（重点）

15min

面向对象

本课的「读代码三步法」

①

大白话翻译

看到一行代码 → 用大白话说清楚「它是干什么的」

②

猜输出

这段代码跑完会得到什么？心里先给个答案

③

问 AI 验证

复制给 AI：「这段代码输出什么？」—— 对照自己的猜测

三步法贯穿全课：每个语法点，都要「翻译 → 猜 → 验证」

二、环境准备 · 三种跑 Python 的方式

方式A · VS Code（推荐）

第1周已装 VS Code

装「Python」扩展 → 新建 01.py

点右上角 ▶ 运行

方式B · 在线环境

免安装，急用时用

online-python.com

pythontutor.com（看逐步动画）

方式C · 官方 IDLE

装 Python 时自带

最简单的交互窗口

适合快速试一行代码

本课怎么用

选一种即可，不纠结

重点是「能跑起来」

后面跟着敲、跟着读

Hello World + 注释

```
01.py
# 这是一行注释，人看、机器不看
print("Hello, World!") # 输出到屏幕

name = "北航"          # 变量：起个名字装数据
print("你好", name)     # 逗号自动加空格

# print 三个用法
print("a", "b", "c")    # 输出：a b c
print("a", "b", "c", sep="-") # 输出：a-b-c
print("换行", end="!")  # 结尾用 !，不换行
```

是注释，解释给人看

- 注释是「旁白」，机器直接跳过
- print() = 把内容打印到屏幕
- 变量 = 给数据起个名字，方便反复用
- sep 改分隔符，end 改结尾

试试：把 "Hello, World!"

改成你的名字，再运行

读报错 · 报错是朋友不是敌人

```
报错信息

Traceback (most recent call last):
  File "01.py", line 1, in <module>
    print(Hello)
NameError: name 'Hello' is not defined
```

读报错只看两行

① 最后一行：错误类型

NameError = 名字没定义

② 带 ↑ 的那一行：

出错的具体位置

常见错误类型：

NameError 未定义 / TypeError 类型错

SyntaxError 语法错 / IndentationError

缩进错

报错 = 机器在告诉你：改这里。先看最后一行，再看箭头指向

三、变量与类型 · 数据的四只「盒子」

```
types.py
age = 20          # int    整数
price = 19.9      # float  小数
name = "张三"     # str    字符串
is_student = True # bool   真假

print(type(age))  # <class 'int'>
print(type(name)) # <class 'str'>

# 变量可重新赋值，类型也会变
age = "二十岁"    # 现在 age 变成 str
```

type() 看类型

- int: 整数, 如 20
- float: 小数, 如 19.9
- str: 字符串, 加引号
- bool: 只有 True / False

变量 = 一个贴了标签的盒子

记忆点: 字符串必须加引号 (其他类型不用)

数字才是不加引号的 20 / 19.9

类型转换 + f-string 拼接

```
convert.py
age = 20
# 数字不能直接和文字相加
# print("我" + age)    # 报错 TypeError

# 转换三兄弟: int() / float() / str()
text = "20"
num = int(text)        # 字符串 → 整数
print(num + 1)         # 21

# f-string: f 开头, { } 里放变量
print(f"我今年{age}岁") # 我今年20岁
```

拼接推荐用 f-string

- int(x) 转整数
- float(x) 转小数
- str(x) 转字符串

f-string 最常用:

f"文字{变量}文字"

花括数字的变量会自动替换

必须先 str() 或 f-string

条件判断 if / elif / else

```
if_demo.py

score = 85

if score >= 90:
    print("优秀")
elif score >= 60:
    print("及格")
else:
    print("不及格")

# 比较运算符
# > >= < <= == !=
# and(且) or(或) not(非)
```

三个关键字

- if: 如果 (第一个条件)
- elif: 否则如果 (可多个)
- else: 否则 (兜底)

== 是比较相等, = 是赋值

~~千万别搞混~~

猜: `score = 59` 输出什么?

`score = 90` 输出什么? 组合多个条件

条件块从 if 开始, 依次判断, 命中一个就停

缩进是 Python 的灵魂 + 实操1

缩进对比

```
if True:
    print("属于 if")      # 缩进 4 空格
print("不属于 if")      # 顶格，一定执行

# 错误示范（少了缩进会报错）
if True:
print("这行会报错 IndentationError")
```

缩进 = 归属关系

- 冒号：后面要另起一行并缩进
- 缩进的代码「属于」上面的块
- 同一块缩进必须对齐
- 推荐统一用 4 个空格

【实操1】写一个分数判断：

输入 score, >60 输出「及格」

否则输出「不及格」，用上 if/else

看到冒号：就想到「接下来要缩进」

四、列表 list · 排队盒子

```
list_demo.py

fruits = ["苹果", "香蕉", "橙子"]

print(fruits[0])      # 苹果 (从 0 开始数)
print(fruits[-1])     # 橙子 (-1 是最后一个)

fruits.append("葡萄") # 往尾巴加一个
print(len(fruits))    # 4 (长度)

# 切片: 取一部分 [起点:终点]
print(fruits[0:2])    # ['苹果', '香蕉']
```

下标从 0 开始

- 列表用 [] 包住, 逗号分隔
- 下标: 0 是第一个, -1 是最后一个
- append() 往尾巴添加
- len() 看长度
- 切片 [0:2] 取第0到第1个
(含头不含尾)

比喻: 列表 = 有编号的抽屉

每个抽屉能装任意数据

for 循环 + 列表推导式

```
for_demo.py

fruits = ["苹果", "香蕉", "橙子"]

# for 循环: 挨个拿、挨个处理
for f in fruits:
    print(f)          # 依次打印三个水果

# range(): 生成一串数
for i in range(5):
    print(i)          # 0 1 2 3 4

# 列表推导式: 一行生成新列表
nums = [x * 2 for x in range(5)]
print(nums)          # [0, 2, 4, 6, 8]
```

for = 循环处理

- for x in 列表:
 逐个取出, 缩进块里处理
- range(n): 0 到 n-1 的数
- 列表推导式:
 [表达式 for x in 列表]

~~一行顶一个 for 循环~~

AI 代码里 for + 推导式极常见
看到能认出即可

for 循环 = 流水线: 同一件事, 对每个元素都做一遍

字典 dict · 带标签的盒子

```
dict_demo.py

student = {
    "name": "张三",
    "age": 20,
    "score": 85
}

print(student["name"])    # 张三（用键取值）

student["city"] = "北京"  # 新增一个键值对
print(student.keys())     # 所有键
```

```
# 用 get 更安全，键不存在不报错
print(student.get("sex", "未知")) # 未知
```

键值对 key: value

- 字典用 {} 包住
- 每个元素是 键: 值
- 靠「键」取值，不靠位置
- 新增: dict["新键"] = 值
- get() 取不到给默认值，更安全

比喻：字典 = 通讯录

按「名字」找人，不按「排第几」

组合拳：遍历「字典列表」 + 实操2

```
combo.py

students = [
    {"name": "张三", "score": 85},
    {"name": "李四", "score": 92},
    {"name": "王五", "score": 58},
]

# 列表里套字典，逐个取出处理
for s in students:
    name = s["name"]
    score = s["score"]
    if score >= 60:
        print(f"{name} 及格")
    else:
        print(f"{name} 不及格")
```

最实用的组合

列表 + 字典 + for + if

是 AI 处理数据的「标准模板」

看到 for s in xxx:

心里默念「挨个处理每个元素」

【实操2】把上面代码里

print 改成 f"姓名: {name}, 成绩: {score}"

五、函数 def · 打包一段操作

```
func_demo.py

# 定义: def 函数名(参数):
def greet(name):
    """这个函数用来打招呼"""
    return f"你好, {name}"

# 调用
msg = greet("张三")
print(msg)                # 你好, 张三

# 多个参数 + 默认值
def add(a, b=10):
    return a + b

print(add(3))              # 13 (b 用默认值 10)
print(add(3, 5))          # 8
```

def + return

- def 定义函数, 之后反复调用
- 参数 = 传给函数的「输入」
- return = 把结果「交回去」
- 默认值: 调用时不传就用它
- 三引号 """ 是函数说明 (docstring)

**看到 def: 把这段代码当成
一个「有名字的黑盒子」**

函数 = 可复用的积木, 看懂输入输出即可, 不必抠内部

参数的花样 · *args 与 **kwargs

```
args_kwargs.py

def demo(a, *args, **kwargs):
    print("a =", a)
    print("args =", args)      # 多余的位置参数
    print("kwargs =", kwargs)  # 多余的键值参数

demo(1, 2, 3, name="张三", age=20)
# a = 1
# args = (2, 3)
# kwargs = {'name': '张三', 'age': 20}
```

看到 * 和 ** 别慌

· *args:

装多余的位置参数 → 元组

· **kwargs:

装多余的键值参数 → 字典

~~AI 框架里到处是 **kwargs~~

只要知道「在收一堆参数」即可
不用背写法，看到能认出

「这是在收集灵活参数」就够了

模块 import · 搬别人写好的代码

```
import_demo.py

import math

print(math.sqrt(16))    # 4.0 (开平方)

from math import pi
print(pi)               # 3.14159...

import json
data = '{"name": "张三"}'
obj = json.loads(data)  # 字符串 → 字典
print(obj["name"])     # 张三
```

两种导入方式

- import 模块:
用 模块.函数 调用
- from 模块 import 名字:
直接调用

json 模块超常用:

loads() 字符串 → 对象
os 文件系统 / sys 系统 / json 数据

dumps() 对象 → 字符串
time 时间 / requests 网络

import = 借用别人造好的轮子，不用自己重写

pip 安装第三方库

终端命令

安装（在终端里敲，不是 .py 里）

```
pip install requests
```

指定版本

```
pip install requests==2.31.0
```

查看已安装

```
pip list
```

卸载

```
pip uninstall requests
```

pip = 装库的工具

- 标准库：自带，import 就能用
- 第三方库：先 pip install 再 import
- 装在「终端」里，不是代码里

VS Code 里：

菜单 终端 → 新建终端 → 敲命令

报错 ModuleNotFoundError?

= 没装这个库，先 pip install

缺啥装啥：看到 ModuleNotFoundError 就去 pip install

六、面向对象 · 为什么要认识「类」

现实世界 → 代码世界

- 类 class = 设计图 / 模板
- 对象 = 按图造出来的具体东西
- 属性 = 它的数据 (姓名、年龄)
- ~~方法 = 它能做的事 (自我介绍)~~

为什么必须认识类?

因为 LangChain / CrewAI 的
Agent、Tool 都是「类」写出来的

```
类的最简示例

class Dog:
    def __init__(self, name):
        self.name = name      # 属性

    def bark(self):           # 方法
        print(f"{self.name}: 汪汪!")

d = Dog("旺财")              # 造出一个对象
d.bark()                     # 旺财: 汪汪!
```

class Dog 是「模板」

d = Dog("旺财") 是「造出一只狗」

self 代表「当前这个对象自己」

类的标准长相 · 四看

看类就找这四样

- ① `__init__`: 初始化函数
造对象时自动跑
- ② `self.xxx`: 属性 (数据)
- ③ `def` 方法: 能做的事
- ④ 类名(参数): 造对象

看到 `__init__` 就想到:
双下划线 `__init__` 是特殊方法
「这是对象的出厂设置」
不是写错了, 别怕

book_class.py

```
class Book:
    def __init__(self, title, author):
        self.title = title    # ① 属性
        self.author = author

    def info(self):           # ② 方法
        return f"{self.title} by {self.author}"

b = Book("三体", "刘慈欣")
print(b.info())    # 三体 by 刘慈欣
```

继承 · 子类复用父类

```
inherit.py

class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name} 在叫")

# 子类继承父类: class 子类(父类)
class Dog(Animal):
    def speak(self):          # 覆盖父类方法
        print(f"{self.name}: 汪汪!")
```

```
d = Dog("旺财")
d.speak()    # 旺财: 汪汪!
```

class 子类(父类)

- 继承 = 儿子继承老子的家产
- 子类自动拥有父类的方法
- 覆盖: 子类重写同名方法
- 扩展: 加自己独有功能

看到 `class X(Y)`.

~~Y 是 X 的一种特殊版本~~
AI 框架常见: 自定义 Tool 类
继承框架的 BaseTool

继承 = 复用父类 + 按需改造

装饰器 @ 与类型注解

```
decorator.py

# 装饰器：给函数「套一层壳」
def log(func):
    def wrapper(*args, **kwargs):
        print("开始调用")
        return func(*args, **kwargs)
    return wrapper

@log
def hello(name):
    print(f"你好 {name}")

hello("张三")    # 先打印"开始调用"再"你好 张三"
```

@ 开头 = 装饰器

- @xxx 在函数上面 = 给函数加功能
- 看不懂没关系，先知道「在装饰」

类型注解（新版常见）：

~~def add(a: int, b: int) -> int:~~

~~a: int 表示 a 是整数~~
看到 -> 别当成箭头

~~-> int 表示返回整数~~
是「函数返回什么类型」

类型注解只是提示，不强制

【实操3】读一段类代码，答三问

```
quiz.py

class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def is_pass(self):
        return self.score >= 60

    def level(self):
        if self.score >= 90:
            return "优秀"
        elif self.score >= 60:
            return "及格"
        return "不及格"

s = Student("小明", 75)
print(s.is_pass())    # ?
print(s.level())      # ?
```

三个问题

Q1: 这个类叫什么?

Q2: 有哪些属性和方法?

Q3: `s.is_pass()` 和 `s.level()`

分别输出什么?

答案:

Q3 → `is_pass()` 输出 `True`

`level()` 输出「及格」

能答出这三问，读类能力就达标了

七、Skills 开发实战 · 先回顾 Skills 是

一句话记住

SKILL.md 是「说明书」

告诉 AI:

- 什么时候用我
- 怎么一步步干活
- 输出成什么样子

辅助脚本 = 真正干活的工具

本课程实战: 让 AI 不只是动嘴
亲手写一个 `python-code-reader`

技能包 (30 分钟)

```
Skills 的结构

Skills (技能包) = 给 AI 的"岗位说明书"包
├── SKILL.md ← 说明书: 教 AI 怎么完成一类任务
└── (可选) 辅助文件: 脚本、模板、数据

放哪里?
  项目级: .claude/skills/<技能名>/SKILL.md
           (跟着项目走, 推荐先用这个)
  个人级: ~/.claude/skills/<技能名>/SKILL.md
           (所有项目都能用)
  CodeBuddy 对应: .codebuddy/skills/ 或
                  ~/.codebuddy/skills/
```

实战任务：做一个「Python 代码讲解助手」

任务目标

做一个叫 python-code-reader 的 Skill，

让 AI 用大白话讲清任意 Python 代码。

四个步骤 · 总共约 30 分钟

1

创建目录（1 分钟）

建 .claude/skills/python-code-reader/ 文件夹

2

写 SKILL.md（15 分钟）

写 description + 5 步执行流程 + 示例对话

3

写 analyze.py 辅助脚本

用今天学的语法，统计代码行数/函数数/类数

4

测试 Skill（4 分钟）

粘贴代码让 AI 讲，看是否触发技能

今天学的所有语法，都会在写这个 Skill 时用上

步骤1+2 · 建目录 & 写 SKILL.md 头部

① 在项目文件夹里创建:

```
目录结构  
.  
claude/skills/python-code-reader/SKILL.md
```

② SKILL.md 开头要有 frontmatter:

--- 包裹的 name 和 description

name: 技能名

description: 什么场景下触发

黄金法则: description 写清「什么时候用」

AI 才会自动触发这个技能

```
SKILL.md (头部)  
  
---  
name: python-code-reader  
description: 用大白话讲解 Python 代码。  
            当用户粘贴 Python 代码、问  
            "这段代码干什么/怎么运行/  
            哪里看不懂"时使用。  
---  
  
# Python 代码讲解助手  
  
## 你负责的任务  
把用户给的 Python 代码，按固定流程讲  
清楚，目标是让零基础的人听懂。
```

frontmatter 的 description 决定 AI 会不会主动用这个技能

SKILL.md 核心 · 五步执行流程

●●● 执行步骤（必须按顺序）

第1步：一句话概括

用不超过 30 个字，说明整体在干什么。

示例："这段代码统计学生成绩，输出及格名单。"

第2步：拆块讲解

每块用：

- 【代码】贴出原代码
- 【干什么】大白话解释
- 【关键词】标注语法点（变量/列表/字典/函数/类/装饰器等），每个一句话解释

第3步：数据流

用箭头说明数据从哪来、怎么处理、到哪去。

示例：输入名单 → for循环判断 → 及格者加入新列表 → 打印

第4步：猜输出

要点：执行步骤写清「先做什么后做什么」，示例对话给 AI 打样——有章法 + 有样例，AI 才能稳定输出

第5步：常见坑

1-2 句提示新手易错处（索引从0开始、==才是判断相等、缩进错误等）。

●●● 示例对话

用户：

```
def add(a, b): return a + b
```

助手：

- 【干什么】定义一个加法函数：输入两个数，返回它们的和
- 【关键词】def=定义函数；return=返回结果
- 【数据流】a、b → 相加 → 返回给调用者
- 【猜输出】print(add(3, 4)) → 7

description 管「触发」，执行步骤管「章法」，示例对话管「打样」

步骤3 · 写 analyze.py 辅助脚本（加分项）

```
.claude/skills/python-code-reader/analyze.py

# analyze.py — 统计一段 Python 代码的基本信息
import sys

def analyze(code):
    lines = code.split("\n")
    non_empty = [l for l in lines if l.strip()] # 去掉空行
    funcs = [l for l in non_empty
              if l.strip().startswith("def ")]
    classes = [l for l in non_empty
               if l.strip().startswith("class ")]

    print("代码总行数:", len(lines))
    print("非空行数:", len(non_empty))
    print("函数数量:", len(funcs))
    print("类数量:", len(classes))
    for f in funcs:
        print("  函数:", f.strip().split("(")[0]
              .replace("def ", ""))

if __name__ == "__main__":
    code = sys.stdin.read() # 读取传入的代码
    analyze(code)
```

今天学的语法全用上了

- import sys: 导入标准库
- def analyze(): 函数
- code.split(): 字符串切分
- 列表推导式: 过滤空行
- startswith(): 判断开头
- if __name__: 入口判断

写进 SKILL.md 执行步骤:

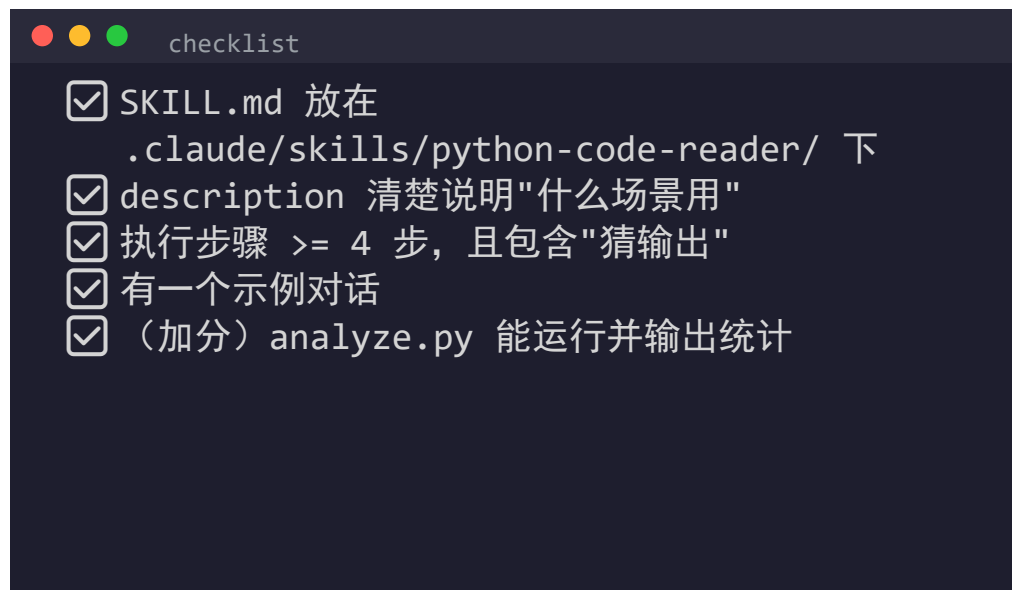
这段代码 = 本课知识的浓缩
「先运行 analyze.py 传代码,
把统计结果当讲解开头」

步骤4 测试 + 验收标准

测试 Skill（4 分钟）

- ① 写一段今天练习的代码
- ② 对 AI 说「帮我看看这段代码是干什么的」+ 粘贴代码
- ③ AI 用了 python-code-reader 的格式讲解 → 成功!
- ④ 没触发? 检查 description 是否写清场景, 或手动指定技能

验收标准（都满足 = 完成）



做完即验收：五条全勾，技能包就立起来了

Skills 开发的三条心法

①

从自己的痛点出发

你最常让 AI 干什么？把它做成 Skill。自己用得上，才写得动

②

description 是灵魂

写不清「什么时候用」，AI 就不会主动用 —— 触发全靠它

③

先小后大

先做一个 10 行的 Skill，跑通流程，再慢慢加功能

本堂的 Skill 是「样板」：课后为自己最常用的一件事各写一个（读论文 / 写周报 / 改简历）

先跑通，再优化 —— 别在第一步就追求完美

八、本课知识地图 · 一张图记住

词

词汇表

变量 = 存数据；列表 = 排队；字典 = 名片夹

流

流程

条件 if/else（做选择）；循环 for（逐个处理）

木

积木

函数 def（封装逻辑）；import（用现成库）

骨

骨架

类 class（图纸）+ self + __init__ + 继承 + @装饰器

武

武器

Skills：把「读代码三步法」做成 AI 技能包

读代码所需的一切 = 词汇 + 流程 + 积木 + 骨架 + 武器

课后作业

1

读代码练习

把本课 4 个实操代码，每段用「三步法」（翻译 → 猜输出 → 验证）过一遍

2

Skills 实战（核心作业）

完善 python-code-reader，再仿照它写 1 个新 Skill（论文阅读/周报/简历），含 description + 至少 3 步

3

预习

GitHub 搜 flask / fastapi 小项目，找一段真实代码，下节课带来 —— 课3 要用它实战阅读

FAQ · 常见疑问

Q1 两小时语法会不会太少？

不会，目标是「读懂」不是「熟练写」，
2 小时已覆盖 90% 读代码需求。

Q2 连 self 都没完全懂能继续吗？

能，记住 self = 「对象自己」即可，
剩下的读项目时自然就懂。

Q3 和第1周 Skills 有啥区别？

第1周「会用现成」；本课「第一次自己写」。

Q4 为什么 3 堂语法压成 1 堂？

因为目标是「读懂」，细节放项目里
「遇到再学」——项目是最好的老师。

Q5 写 analyze.py 卡住怎么办？

- ① 对照 7.2 步骤3 抄一遍
- ② 看不懂的复制给 AI 解释
- ③ 报错了把报错发给 AI

这不是作弊，是这门课的核心工作方式。

读懂 · 会问 · 能开发

今天：认了 Python 的词汇、流程、积木、骨架
亲手造出第一个 Skills 技能包

下节预告

课2 · Python 数据分析：读代码 + 处理表格数据
NumPy / Pandas / Matplotlib —— 第3周机器学习的直接基础

课后把「读代码三步法」变成肌肉记忆，我们下节课见